

UPS Address Validation API Setup Guide

Replacing the built-in residential / commercial decision with the UPS address classification service

Purpose	Deploying the proxy, configuring credentials, verifying it, the interface contract, client setup, and known limits
Audience	Administrators (sections 1–4, 6–7); everyday users (section 5)
Prerequisites	A UPS developer account, access to a Supabase project, the Supabase CLI (section 2)
Project ref	Your own Supabase project Reference ID (how to find it: 2.2). Written throughout as <your-project-ref>
Proxy source	<code>supabase/functions/ups-address/index.ts</code>

▪ Contents

1 Overview	What it does, how the pieces fit, where the credentials live
1.1 What it does	
1.2 Architecture	
1.3 Where credentials live	
2 Prerequisites	Three things you need, where each comes from, how to confirm it
2.1 UPS developer account	
2.2 Supabase project access	
2.3 Supabase CLI	
2.4 Readiness checklist	
3 Deployment	Get credentials, link the project, set the variables, deploy
3.1 Obtain UPS API credentials	
3.2 Link your Supabase project	
3.3 Set the environment variables	
3.4 Deploy the function	
4 Verification	Confirm deployment, authentication and the endpoint, layer by layer
4.1 Confirm the deployment	
4.2 Get an access token	
4.3 Confirm the endpoint	
4.4 Switch to production	
5 Client setup	Choosing the classification source and entering the endpoint
6 Interface reference	Request and response, client behaviour, the proxy's call to UPS
6.1 Request	
6.2 Response	
6.3 Client behaviour	
6.4 The proxy's call to UPS	
7 Known limits and what to weigh	What is not supported today, per-account credentials, how to trial it
7.1 Local-account mode is not supported	
7.2 Per-account credentials	
7.3 Before you turn it on	
8 Appendix — troubleshooting	On-screen messages, status codes at a glance
8.1 Status codes at a glance	

1 Overview

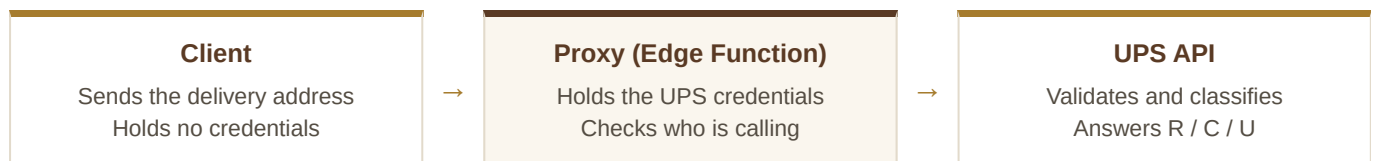
1.1 What it does

By default the system decides whether each shipment is residential or commercial from the billing codes, the service name and the invoice history. With this feature on, the delivery address is sent to the official UPS address validation service instead, which answers residential or commercial.

Residential and commercial rates differ, so that answer changes the money. Read section 7 before turning it on.

1.2 Architecture

The browser cannot call UPS directly: UPS requires OAuth and does not allow cross-origin access (CORS). A proxy is therefore deployed as a Supabase Edge Function, holds the credentials, and makes the call on the browser's behalf.



The answers are **R** residential, **C** commercial, **U** undetermined. On **U**, or on any failure, pricing falls back to the built-in logic and carries on — nothing stalls.

1.3 Where credentials live

The UPS Client ID and Client Secret exist only as environment variables on the proxy. The client settings screen has no field for them, deliberately.

Why there is no field

Every field on the settings screen is written into `CFG.raw`, and that object goes into browser `localStorage`, into the account's synced settings row, and into any exported settings file. A credential placed there would spread through all three. On top of that, CORS stops the browser from reaching UPS at all, so a credential entered there could not be used even if it were safe to store.

2 Prerequisites

Three things are needed before section 3. They come from different places and serve different purposes; without any one of them, deployment cannot finish.

2.1 UPS developer account

What it is for: obtaining the Client ID and Client Secret that authorise calls to UPS, and subscribing to the Address Validation product. UPS issues credentials per application, and one set of credentials draws on one UPS account's quota.

Item	Detail
Where	<code>developer.ups.com</code> → Apps → Add App
Requires	A working UPS account number — one that actually ships and is invoiced
Product to add	Address Validation . Without it the credentials are still valid, but calls come back <code>502 UPS 403</code>
What you get	A Client ID and a Client Secret. The Secret is shown once, at creation — save it then
Environments	Test <code>wwcie.ups.com</code> , production <code>onlinetools.ups.com</code> . Both use the same credentials

One UPS account, one set of credentials

Usage is billed to the UPS account the credentials belong to. Several UPS accounts each need their own application and their own credentials; sharing one set makes quota and billing impossible to attribute.

2.2 Supabase project access

What it is for: the proxy runs as an Edge Function on a Supabase project, and that project holds the UPS credentials.

Item	Detail
Role needed	Owner or Developer on the project. Read access cannot deploy functions or write environment variables
Project ref	The Reference ID under Settings → General — the subdomain in your endpoint URL. Commands in this guide write it as <code><your-project-ref></code> ; substitute your own
Key you need	The anon (publishable) key, for the test in 4.2. This key is public by design and ships in your <code>auth-config.js</code> ; what protects the data is row-level security and function grants, not the key's secrecy. Never substitute the <code>service_role</code> key
How to check	If Edge Functions appears in the console sidebar, your role is sufficient

2.3 Supabase CLI

What it is for: deploying the function and writing environment variables. Neither can be done from the web console; the CLI is required.

Platform	Install
macOS	<code>brew install supabase/tap/supabase</code>
Windows	<code>scoop install supabase</code>
Any (Node)	<code>npm install -g supabase</code>

Check the version and sign in:

```
supabase --version
supabase login
```

`supabase login` opens a browser to authorise; afterwards the CLI acts with that account's permissions. Once per computer.

2.4 Readiness checklist

All four must hold before section 3.

#	Item	How to confirm
1	The UPS application exists and subscribes to Address Validation	The product is listed on the app's detail page and shows as approved
2	The Client ID and Client Secret are saved	You can read both back. A lost Secret must be regenerated
3	You can deploy to the Supabase project	Edge Functions is visible in the console
4	The CLI is installed and signed in	<code>supabase --version</code> prints something and <code>supabase login</code> has completed

3 Deployment

3.1 Obtain UPS API credentials

- 1 Create an application at `developer.ups.com`.
- 2 Add **Address Validation** to that application's products.
- 3 Take the Client ID and Client Secret.

The product subscription is not optional

The proxy calls `POST /api/addressvalidation/v2/3`. The trailing `3` is request option 3 — validation *and* classification. An application subscribed only to Rating or Tracking does not cover it, and verification will return `502 UPS 403`.

3.2 Link your Supabase project

```
# authorise the CLI (opens a browser)
supabase login

# from the repository root; writes supabase/config.toml
supabase link --project-ref <your-project-ref>
```

The repository ships without `config.toml`, so linking comes before the first deploy.

3.3 Set the environment variables

```
supabase secrets set \
  UPS_CLIENT_ID=<client-id> \
  UPS_CLIENT_SECRET=<client-secret> \
  UPS_BASE=https://wwwcie.ups.com
```

Variable	Detail
UPS_CLIENT_ID	Required
UPS_CLIENT_SECRET	Required
UPS_BASE	Optional. Defaults to <code>https://onlinetools.ups.com</code> (production). Point the first deployment at <code>https://wwwcie.ups.com</code> (the CIE test environment) so it spends no production quota
SUPABASE_URL SUPABASE_ANON_KEY	Do not set these — the platform injects them

3.4 Deploy the function

```
supabase functions deploy ups-address
```

Changing an environment variable takes effect immediately; no redeploy needed.

4 Verification

The proxy answers different failures with different status codes, so working through them in order tells you which layer is wrong. Run 4.1 to 4.3 in sequence; do not skip ahead.

4.1 Confirm the deployment

Call it with no credentials at all:

```
curl -i -X POST \
  "https://<your-project-ref>.supabase.co/functions/v1/ups-address" \
  -H "Content-Type: application/json" -d "{}"
```

Response	Reading
401 not signed in	Deployed. The proxy is correctly refusing an unauthenticated request
404	Not deployed — back to 3.4

4.2 Get an access token

In the Supabase console, Authentication → Users → Add user, create a test account (email and password, marked confirmed). Then:

```
curl -X POST \
  "https://<your-project-ref>.supabase.co/auth/v1/token?grant_type=password" \
  -H "apikey: <anon key, see auth-config.js>" \
  -H "Content-Type: application/json" \
  -d '{"email": "<email>", "password": "<password>"}'
```

Take `access_token` from the response.

4.3 Confirm the endpoint

```
curl -X POST \
  "https://<your-project-ref>.supabase.co/functions/v1/ups-address" \
  -H "Authorization: Bearer <access_token>" \
  -H "Content-Type: application/json" \
  -d '{"address": "1600 Pennsylvania Ave NW", "city": "Washington",
    "state": "DC", "postal": "20500", "country": "US"}'
```

Response	Reading and what to do
200 {"class": "R" "C" "U"}	Working. Continue to 4.4
500 UPS credentials not configured	Environment variables not set — back to 3.3
502 UPS 401	Wrong UPS credentials — check the Client ID and Secret
502 UPS 403	The application does not subscribe to Address Validation — back to 3.1

To read the logs:

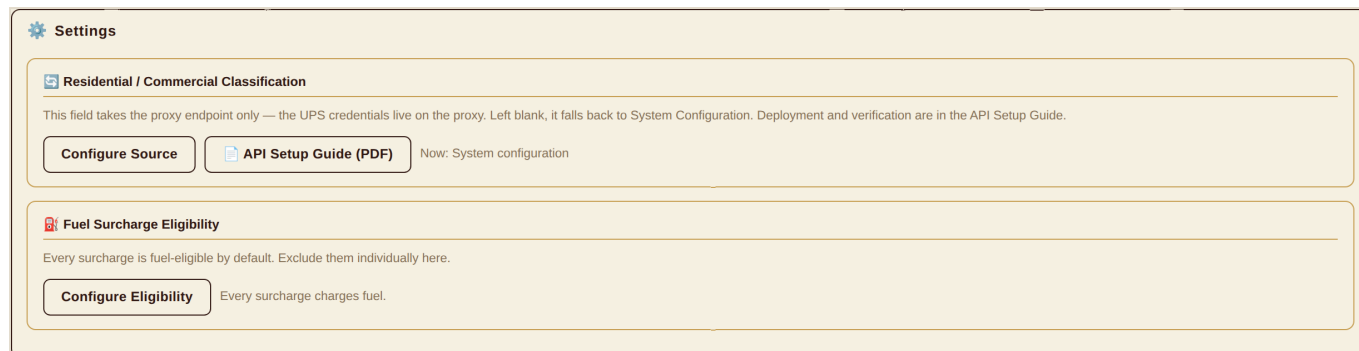
```
supabase functions logs ups-address
```

4.4 Switch to production

```
supabase secrets set UPS_BASE=https://onlinetools.ups.com
```

5 Client setup

Where: the Settings tab → Residential / Commercial Classification.

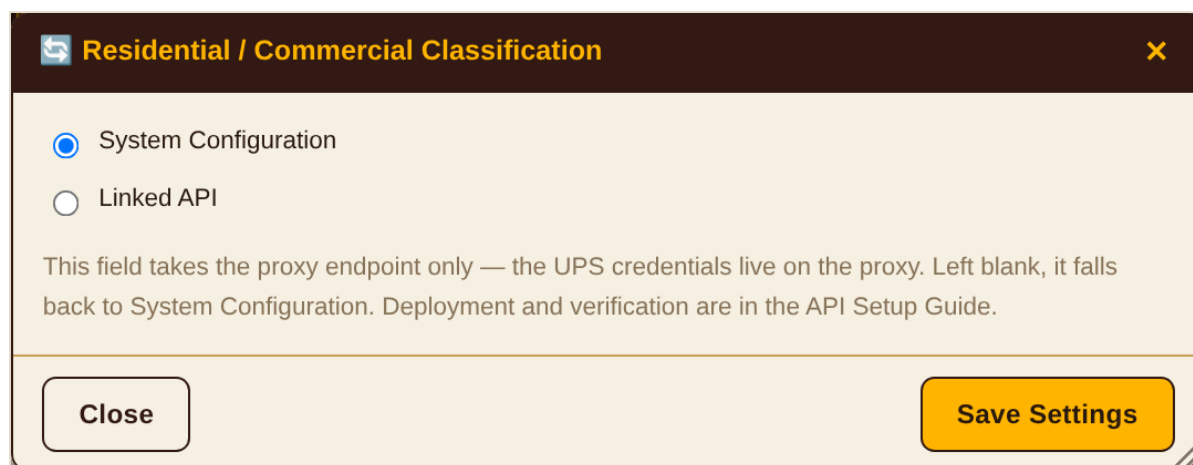


The screenshot shows the 'Settings' tab with a gear icon. Below it is the 'Residential / Commercial Classification' section, which has a sub-header and a descriptive text: 'This field takes the proxy endpoint only — the UPS credentials live on the proxy. Left blank, it falls back to System Configuration. Deployment and verification are in the API Setup Guide.' There are two buttons: 'Configure Source' and 'API Setup Guide (PDF)'. To the right of the second button is the text 'Now: System configuration'. Below this is the 'Fuel Surcharge Eligibility' section, with a sub-header and text: 'Every surcharge is fuel-eligible by default. Exclude them individually here.' It has a 'Configure Eligibility' button and the text 'Every surcharge charges fuel.'

Figure 5-1 The Settings tab. Residential / Commercial Classification is the block this guide concerns; it is configured and saved inside its own panel — there is no shared save button for the page.

Where this document comes from

The “**API Setup Guide (PDF)**” button on the same row opens this file. It is `UPS-API-Setup-Guide-EN.pdf` (the Chinese edition is `UPS-API-Setup-Guide.pdf`), and it must sit in the same folder as `index.html`. If it was not deployed alongside, the button says where the file belongs rather than doing nothing.



The screenshot shows a modal window titled 'Residential / Commercial Classification' with a close button (X) in the top right. Inside, there are two radio buttons: 'System Configuration' (selected) and 'Linked API'. Below them is the same descriptive text as in Figure 5-1. At the bottom, there are two buttons: 'Close' and 'Save Settings'.

Figure 5-2 The default is System Configuration — billing codes, service name and invoice history.

 Residential / Commercial Classification 

☐ System Configuration

☒ Linked API

API proxy endpoint (Edge Function URL)

This field takes the proxy endpoint only — the UPS credentials live on the proxy. Left blank, it falls back to System Configuration. Deployment and verification are in the API Setup Guide.

Close

Save Settings

Figure 5-3 Choose Linked API, paste the endpoint you deployed in 3.4, then Save Settings. This field takes the proxy URL only; the UPS credentials live on the proxy (see 1.3).

How the panel behaves

A change inside the panel applies to the in-memory settings the moment you click, and pricing reflects it at once — but nothing is written yet. **Close discards and reverts; Save Settings writes and closes.**

The summary line shows what is actually in force

If Linked API is selected but the endpoint is blank, both the block summary and the radio show System Configuration — because with no endpoint there is nothing to call, and the built-in logic is what actually runs. Showing the setting rather than the behaviour would let someone believe the API was live when it was not.

6 Interface reference

This section defines the proxy endpoint, for anyone integrating against it directly or debugging at the request level. Ordinary setup does not require it.

6.1 Request

Item	Value
Method	POST. Anything else returns 405 ; OPTIONS serves the CORS preflight
Path	/functions/v1/ups-address
Required headers	Authorization: Bearer <access_token> Content-Type: application/json
CORS	Origin *, headers authorization, content-type

Body fields follow. At least one of `address` or `postal` is required, otherwise the call returns 400 .

Field	Type	Detail
address	string	Street address, first line
city	string	City
state	string	State code
postal	string	Postal code; the proxy takes the first five characters
country	string	Country code, defaults to US
name	string	Recipient name, optional; becomes UPS's ConsigneeName

6.2 Response

```
{ "class": "R" | "C" | "U", "description": "<UPS classification text>" }
```

class	UPS code	Meaning
R	AddressClassification.Code = 2	Residential
C	AddressClassification.Code = 1	Commercial
U	AddressClassification.Code = 0 or absent	UPS could not tell

Failures return `{"error": "<message>"}` ; status codes are in 8.1. A 502 also carries `detail` , the first 300 characters of the UPS response body.

The classification arrives in one of two places

UPS's XAVResponse may put AddressClassification at the top level, or under the first Candidate. The proxy reads both, in that order, so a difference in placement never turns into a U .

6.3 Client behaviour

The client calls this endpoint before each repricing run. It behaves as follows.

Item	Behaviour
Deduplication	Addresses in a batch are normalised and grouped; identical addresses are called once
Concurrency	Six at a time, to stay under the proxy's and UPS's rate limits
Cache	Successful answers are kept, keyed by address; later runs do not call again
Failures	Failures are never cached. That run falls back to the built-in logic; once the configuration is fixed, the next run retries automatically
No endpoint set	No request is sent at all. Everything falls back to the built-in logic, and the screen says the endpoint is blank

Why failures must not be cached

If a failed call were recorded as `U`, then a mistyped endpoint, an undeployed function or a missing permission would be written down permanently as “cannot tell” — and fixing the configuration would not undo it, because nothing would retry. Clearing it would mean wiping browser storage by hand. That behaviour is ruled out deliberately.

6.4 The proxy's call to UPS

Item	Value
Endpoint	<code>{UPS_BASE}/api/addressvalidation/v2/3</code>
requestoption	3 (Address Validation + Address Classification)
Authorisation	<code>client_credentials</code> , via <code>/security/v1/oauth/token</code>
Token reuse	Held at the function's module level and reused until 60 seconds before expiry
Trace headers	<code>transId</code> (a UUID per request) and <code>transactionSrc: ups-reprice</code>

7 Known limits and what to weigh

7.1 Local-account mode is not supported

As currently deployed, the endpoint cannot work

The proxy uses `auth.getUser()` to confirm the caller is a real Supabase user. The system runs in local-account mode, which creates no Supabase session, so every request returns `401` and pricing falls back to the built-in logic in all cases.

This does not make sections 3 and 4 pointless: what they verify is the proxy and the UPS credentials, which are independent of how the client authenticates. Once done, that infrastructure is there and reusable. Section 5's setting only takes effect after the authentication model is switched and the accounts migrated.

7.2 Per-account credentials

If each user must draw on their own UPS quota, the proxy has to look the credentials up by caller: keep them in a table only the server can read, and have the proxy fetch the row belonging to the authenticated user. The endpoint URL stays shared. This too depends on there being a real user identity.

7.3 Before you turn it on

The API outranks every other source

Inside `priceShipment()`, an answer from the API takes precedence over billing codes and invoice history. Any shipment the API can classify is classified by the API — including shipments whose invoice already carries a residential code.

So what changes is not a handful of previously ambiguous shipments; it is every shipment the API can classify, and the priced total moves with them.

Suggested approach: turn it on for one account first. Reprice the same invoice period with it on and with it off, compare the totals and the residential / commercial counts, and see how often UPS's classification disagrees with the billing codes. Then decide whether to roll it out.

8 Appendix — troubleshooting

On-screen message	Cause and what to do
"Linked API" is selected with no proxy endpoint	The endpoint field is blank — see section 5
API proxy call failed (HTTP 401)	No valid authenticated session — see 7.1
API proxy call failed (HTTP 404)	Wrong endpoint URL, or the function is not deployed — see 3.4
API proxy call failed (HTTP 500)	UPS credentials not set on the proxy — see 3.3
N addresses failed	Some requests failed; those fall back to the built-in logic. Failures are not cached, so the next run retries them
The API returned no classification	UPS answered U (cannot tell) for every address in the batch
"Linked API" is selected but the panel still shows System Configuration	The endpoint is blank; the panel shows what is actually in force — see section 5

8.1 Status codes at a glance

Status	Layer	Go to
404	Function not deployed	2.4
401	Caller not authenticated	7.1
500	UPS credentials not set on the proxy	2.3
502 UPS 401	Wrong UPS credentials	2.1
502 UPS 403	UPS product not subscribed	2.1
200	Normal	—